

PROJECTFICHE – Mobiliteit rond de school

Inhoud

1	Situering.....	2
2	Doelstellingen van het project	2
3	Technologie stack.....	2
4	Architectuur (verplicht)	3
4.1	MODEL	3
4.2	VIEW	3
4.3	CONTROLLER (uitbreidbaar).....	3
5	Databank (verplicht model)	4
6	Basisfunctionaliteiten (verplicht).....	5
6.1	CRUD-functionaliteit (Create, Read, Update, Delete).....	5
6.2	Analysefunctionaliteiten	6
7	Dashboard functionaliteiten.....	7
7.1	Minimale vereisten.....	7
8	Uitbreidingen.....	9
8.1	Optie 1: CO ₂ - berekening	9
8.2	Optie 2: Logging functionaliteiten	9
8.3	Optie 3: Afwezigheden & verplaatsingsgedrag.....	10
8.4	Optie 4: Route- en reistijdanalyse	10
9	Groepswerking (3–4 leerlingen)	12
10	Oplevering.....	12

1 Situering

Je ontwikkelt een desktopapplicatie die mobiliteitsdata van leerlingen analyseert. Je werkt volgens de Model–View–Controller (MVC) architectuur met Python, Tkinter en SQLite.

2 Doelstellingen van het project

Je leert in team:

- ✓ de principe van agile/scrum toepassen binnen de context van je project
- ✓ een gestructureerde applicatie bouwen volgens MVC
- ✓ data opslaan in SQLite
- ✓ CRUD-operaties uitvoeren
- ✓ data analyseren via SQL queries
- ✓ een aantal dashboards bouwen in Tkinter
- ✓ samenwerken via GitHub

3 Technologie stack

- ✓ Python
- ✓ SQLite databank
- ✓ Tkinter GUI
- ✓ MVC architectuur
- ✓ GitHub versiebeheer

De leerkracht voorziet per projectteam een Github project dat jullie dienen te gebruiken doorheen de realisatie van dit project.

4 Architectuur (verplicht)

4.1 MODEL

- ✓ Bevat alle databanklogica.
- ✓ SQLite database
- ✓ CRUD operaties
- ✓ SQL queries

4.2 VIEW

Tkinter interface met:

- ✓ invoerformulieren
- ✓ overzichtstabellen
- ✓ dashboard scherm

De UI dient gebruiksvriendelijk te zijn voor de gebruikers van de toepassing.

4.3 CONTROLLER (uitbreidbaar)

Deze laag bevat de business logica en dient het effectieve gedrag van de toepassing te bepalen:

- ✓ validatie van input
- ✓ berekeningen (bv. CO₂)
- ✓ statistische queries
- ✓ filters en sortering
- ✓ ... (zinvol gedrag in functie van de toepassing)...

Tip: De Controller mag SQL-resultaten combineren en verwerken. Hierdoor kan je ervoor kiezen om complexe JOINS te vermijden en data stap voor stap op te halen uit de databank.

5 Databank (verplicht model)

De databank moet **minstens** bestaan uit volgende 3 tabellen. Denk goed na of je deze wel/niet dient uit te breiden met bijkomende velden/tabellen.

Tabel: Students

- ✓ id
- ✓ naam
- ✓ klas
- ✓ afstand

Tabel: Transport

- ✓ id
- ✓ type

Tabel: Mobility_log

- ✓ id
- ✓ student_id
- ✓ transport_id
- ✓ datum

Je vult de databank op met data uit:

- ✓ De 3 beschikbare .csv bestanden op Smartschool:
 - Students.csv
 - Deze bevat data van 150 fictieve leerlingen
 - De leerlingen zijn verdeeld over 4 klassen (6ADB, 6AC, 6AD, 6AE)
 - Afstanden 0,5 km -> 15 km
 - Transports.csv
 - Fiets
 - Bus
 - Auto
 - Te voet
 - Mobility_logs.csv
 - 600 verplaatsingen
 - Willekeurige data in het jaar 2026
 - Koppeling student - transport

Voorzie een hulpprogramma (python) waarmee je deze data in je databank van je project kan inladen.

- ✓ Vul de data aan met testgegevens over hoe jullie projectteam zich verplaatst van en naar de school (voor elk lid voeg je testdata manueel toe).

Opmerking:

De databank is genormaliseerd. Dit betekent dat gegevens verspreid zijn over meerdere tabellen. Je kan deze gegevens combineren via SQL JOINS, maar ook via meerdere aparte queries in je applicatielogica.

6 Basisfunctionaliteiten (verplicht)

Je applicatie moet gebruikers toelaten om mobiliteitsdata **op te slaan, te beheren en te analyseren**.

Dit bestaat uit twee delen: **CRUD-functionaliteit** en **analysefunctionaliteit**.

6.1 CRUD-functionaliteit (Create, Read, Update, Delete)

Je moet volgende gegevens kunnen beheren via je applicatie:

Studenten beheren

De gebruiker moet:

- ✓ een nieuwe student kunnen toevoegen (naam, klas, afstand tot school)
- ✓ bestaande studenten kunnen bekijken in een overzicht
- ✓ gegevens van een student kunnen aanpassen
- ✓ een student kunnen verwijderen (bewaak de consistentie van alle data in de databank!)

Voorbeeld: “Jan Peeters, 6ADB, woont 5 km van school”

Vervoersmiddelen beheren

De gebruiker moet:

- ✓ vervoersmiddelen kunnen toevoegen (bv. fiets, bus, auto, te voet)
- ✓ een lijst kunnen bekijken van alle bestaande vervoersmiddelen
- ✓ vervoersmiddelen kunnen verwijderen (bewaak de consistentie van alle data in de databank!)

Verplaatsingen registreren

De gebruiker moet:

- ✓ per student een verplaatsing kunnen registreren, aanpassen en verwijderen
- ✓ hierbij kiezen:
 - student
 - vervoersmiddel
 - datum (optioneel: duur van verplaatsing)
- ✓ bestaande verplaatsingen kunnen bekijken in een overzicht

Voorbeeld: “Jan komt op 12/03/2026 met de fiets naar school”

6.2 Analysefunctionaliteiten

Je applicatie moet de opgeslagen data analyseren via SQL-queries. Via deze functionaliteit volstaat het om de uitkomsten te tonen in tabelvorm op het scherm.

Tip: Bouw de logica van de analyses op door gebruik te maken van meerdere eenvoudige SQL-statements in combinatie met Python. Je gebruikt hier geen SQL statements met JOINS in.

Je moet minstens volgende analyses voorzien:

Aantal per vervoersmiddel

Toon:

- ✓ hoeveel verplaatsingen gebeuren per type (fiets, bus, auto...)

Voorbeeld output:

- fiets: 25
- bus: 10
- auto: 8

Gemiddelde afstand

Bereken:

- ✓ de gemiddelde afstand tot school van alle studenten

Je voorziet ook volgende uitbreiding op deze analyse functionaliteit:

- gemiddelde per vervoersmiddel

Overzicht per klas

Toon:

- ✓ aantal studenten per klas
- ✓ hun gemiddelde afstand
- ✓ de verdeling vervoersmiddelen per klas

Voorzie 1 extra analysefunctionaliteit die jullie zelf bedenken en integreren in jullie toepassing.

7 Dashboard functionaliteiten

Het dashboard toont de belangrijkste inzichten uit de data.

Belangrijk principe:

Het dashboard toont samengevatte inzichten, geen volledige databankweergave.

Aanbeveling:

Om het dashboard overzichtelijk te houden, werk je met tabbladen (bijvoorbeeld via ttk.Notebook in Tkinter).

Elk tabblad bevat één logisch onderdeel van de applicatie:

- ✓ Overzicht data
- ✓ Vervoersmiddelen analyse
- ✓ Afstand analyse
- ✓ Klassenanalyse
- ✓ Uitbreidingen (CO₂, logging, ...)

Dit verhoogt de leesbaarheid en zorgt voor een gestructureerde gebruikersinterface.

7.1 Minimale vereisten

Je dashboard bevat minstens:

Tabelweergave

Toon data van één tabel **overzichtelijk** op het scherm:

- ✓ Students
- ✓ Transport
- ✓ Mobility_log

Belangrijk

- ✓ Data komt rechtstreeks uit SQLite (Je kiest via een combobox welke tabel je wil zien op het scherm).
- ✓ Geen hardcoded data
- ✓ Wijzigingen in databank zijn zichtbaar na refresh

Verdeling vervoersmiddelen

Toon hoe verplaatsingen verdeeld zijn over transporttypes.

Voorbeeld:

- ✓ Fiets: 50%
- ✓ Bus: 25%
- ✓ Auto: 20%
- ✓ Te voet: 15%

Vorm (beide vormen mogen onder elkaar op hetzelfde tabblad):

- ✓ tabel
- ✓ eenvoudige grafiek (gebruik een balkdiagram)

Gemiddelde afstand per vervoersmiddel

Toon:

- ✓ gemiddelde afstand van alle studenten
- ✓ gemiddelde afstand per vervoersmiddel

Vorm (de verschillende vormen mogen onder elkaar op hetzelfde tabblad):

- ✓ tabel
- ✓ eenvoudige grafiek (staafdiagram)
- ✓ *Optioneel/Extra:* gestapelde staafdiagram waarbij je een logica voorziet om de afstand op te delen in 3 categorieën zodat je per vervoersmiddel kan zien hoe de afstanden verdeeld zijn:
 - korte ritten (0 – 5 km)
 - middellange ritten (5,1 – 10 km)
 - lange ritten (> 10 km)

Advies:

Voorzie eerst alleen de 2 bovenste vormen. Enkel als er tijd over is, kan je de optionele visualisatie ook proberen toevoegen.

Overzicht per klas

Toon:

- ✓ per klas:
 - aantal leerlingen
 - gemiddelde afstand
 - verdeling vervoersmiddelen

Vorm:

- ✓ Tabel
- ✓ Kleine grafiek per klas (focus bij jullie keuze hiervoor op overzicht, niet op design)

8 Uitbreidingen

Zodra de main de basisfunctionaliteiten bevat in jullie project, maak je een branch aan per lid van jullie projectteam. Deze branch maak je herkenbaar door de naam als volgt vorm te geven ***uitbreiding_voornaam_familienaam*** te gebruiken als naam van deze brach

Iedereen kiest één uitbreiding (elke uitbreiding mag maximaal 1 keer per projectteam opgenomen worden).

Bij deze uitbreiding mag je enkel de code uitbreiden, niet herschrijven!

8.1 Optie 1: CO₂ - berekening

Bereken de impact van vervoersmiddelen. Hierbij gebruik je bijvoorbeeld volgende fictieve CO₂ uitstoten per transportmiddel:

- ✓ Fiets = 0 g CO₂
- ✓ Bus = 50 g/km CO₂
- ✓ Auto = 120 g/km CO₂

Indien je dataset over andere vervoersmiddelen beschikt, dien je te beslissen hoe je hier mee omgaat in deze berekening.

Voorzie de mogelijkheid dat gebruikers de data kunnen filteren:

- ✓ Per klas
- ✓ Per afstand (beperkt de opties bijv >5 km, >10 km)
- ✓ Per vervoersmiddel

Je mag de databank uitbreiden voor deze uitbreiding maar:

- ✓ Je mag bestaande tabellen NIET aanpassen.
- ✓ Je mag dus enkel nieuwe tabellen toevoegen.

Zorg ervoor dat deze uitbreiding zichtbaar gemaakt kan worden via de dashboard functionaliteiten en dit zowel in tabelvorm als in één specifieke visualisatie die je hiervoor het best passend vindt.

8.2 Optie 2: Logging functionaliteiten

Voorzie een systeem dat acties van gebruikers logt, zoals:

- ✓ Inloggen / uitloggen
- ✓ Toevoegen / bewerken / verwijderen van data
- ✓ Bekijken van bepaalde pagina's

Elke log bevat:

- ✓ Gebruiker (user_id)
- ✓ Actietype (login, create, update, delete, view)
- ✓ Tijdstip (timestamp)

De user_id dient niet uit de databank te komen, maar mag je opvragen als je bijvoorbeeld de logging functionaliteit activeert (bijv via een messagebox).

Je mag de databank uitbreiden voor deze uitbreiding maar:

- ✓ Je mag bestaande tabellen NIET aanpassen.
- ✓ Je mag dus enkel nieuwe tabellen toevoegen.

Voorzie de mogelijkheid om volgende 'berekeningen/analyses' uit te voeren op de logs:

- ✓ Aantal acties per gebruiker
- ✓ Aantal acties per type (bv. hoeveel logins vs edits)
- ✓ Meest actieve gebruiker(s)

Zorg ervoor dat deze uitbreiding zichtbaar gemaakt kan worden via de dashboard functionaliteiten en dit zowel in tabelvorm als in één specifieke visualisatie die je hiervoor het best passend vindt.

8.3 Optie 3: Afwezigheden & verplaatsingsgedrag

Breid de applicatie uit met een systeem dat bijhoudt of een student effectief aanwezig was of niet op een bepaalde dag van registratie.

Dit laat toe om het mobiliteitsgedrag te analyseren in relatie tot aanwezigheid.

Van elke afwezigheid registreer je volgende gegevens:

- ✓ Id
- ✓ Student_id
- ✓ Datum
- ✓ Status (aanwezig, afwezig, laat)

Je mag de databank uitbreiden voor deze uitbreiding maar:

- ✓ Je mag bestaande tabellen NIET aanpassen.
- ✓ Je mag dus enkel nieuwe tabellen toevoegen.

Voorzie de mogelijkheid om volgende 'berekeningen/analyses' uit te voeren op de afwezigheden & het verplaatsingsgedrag:

- ✓ Aantal afwezigheden per klas
- ✓ Relatie tussen vervoersmiddel en aanwezigheid
- ✓ Percentage aanwezigheid per klas

Zorg ervoor dat deze uitbreiding zichtbaar gemaakt kan worden via de dashboard functionaliteiten en dit zowel in tabelvorm als in één specifieke visualisatie die je hiervoor het best passend vindt.

8.4 Optie 4: Route- en reistijdanalyse

Breid de applicatie uit met een simulatie van reistijd (travel_time_estimation) en route-efficiëntie op basis van afstand en vervoersmiddel.

Dit maakt de applicatie realistischer als "mobiliteitsanalyse-tool".

Voor elke travel_time_estimation registreer je volgende gegevens:

- ✓ Id
- ✓ Student_id
- ✓ Transport_id

- ✓ Geschatte_reistijd (minuten)
- ✓ datum

Voor de eenvoudigheid mag je werken met vaste snelheden per transportmiddel. Bijvoorbeeld:

- ✓ fiets: 15 km/u
- ✓ bus: 30 km/u
- ✓ auto: 50 km/u
- ✓ te voet: 5 km/u

De reistijd kan je dan berekenen door de afstand te delen door de snelheid.

Voorzie de mogelijkheid om volgende 'berekeningen/analyses' uit te voeren op de travel_time_estimations:

- ✓ Gemiddelde reistijd per vervoersmiddel
- ✓ Klas met de langste reistijd
- ✓ Relatie afstand versus reistijd

Zorg ervoor dat deze uitbreiding zichtbaar gemaakt kan worden via de dashboard functionaliteiten en dit zowel in tabelvorm als in één specifieke visualisatie die je hiervoor het best passend vindt.

9 Groepswerking (3–4 leerlingen)

Iedereen moet elk deel kunnen uitleggen + demonstreren.

GitHub commits moeten individueel zichtbaar zijn + de verwachting is dat je op regelmatige basis zinvolle commits uitvoert met een heldere beschrijving van wat de toegevoegde waarde is van je commit.

Je voorziet een klassendiagram (draw.io) die volledig in lijn is met de opgeleverde code.

De code zelf dient op een leesbare manier opgeleverd te worden (voorzie de nodige commentaar hiervoor, deel de projectstructuur op in subfolders, ...).

Je dient elke week als projectteam de status van je project voor te stellen aan de leerkracht (niet klassikaal). Het is jullie verantwoordelijkheid om de leerkracht hier wekelijks op aan te spreken (geen demo / uitleg gedurende een week waarin effectief aan het project gewerkt werd = geen punten voor het werk van jullie team die week).

10 Oplevering

De finale demo + deadline voor dit project op te leveren is vrijdag 5 juni 2026 via Smartschool.

Volgende deliverables lever je op via de uploadzone in smartschool:

- Zip bestand met daarin alle code
- De SQLite databank
- Het Klassendiagram (draw.io)

Je gebruikt hiervoor de uploadfolder 10. Eindproject MVC – Python – Tkinter:



Het volstaat als één iemand van het projectteam alle bestanden (= alle branches) aanlevert via Smartschool.